

// Jivops

RAG en Producción

Guía completa — 30 lecciones

Arquitectura de RAG · Bases vectoriales · Búsqueda híbrida · Generación y
síntesis · Evaluación y calidad · Producción avanzada

Contenido

Módulo 1: Arquitectura de sistemas RAG

- Lección 01 — Anatomía de un pipeline RAG en producción
- Lección 02 — Ingesta de documentos — fuentes y formatos
- Lección 03 — Parsers y extracción de contenido (PDF, HTML, Office)
- Lección 04 — Preprocesamiento y limpieza de texto
- Lección 05 — Metadata y enriquecimiento de fragmentos

Módulo 2: Bases de datos vectoriales

- Lección 06 — Comparación de bases de datos vectoriales (Pinecone, Weaviate, Qdrant, pgvector)
- Lección 07 — Índices ANN — HNSW y IVF
- Lección 08 — Filtrado por metadata en búsquedas vectoriales
- Lección 09 — Namespaces y multi-tenancy en bases vectoriales
- Lección 10 — Escalado y sharding de índices vectoriales

Módulo 3: Búsqueda híbrida y recuperación avanzada

- Lección 11 — Búsqueda híbrida — combinar vectorial y léxica (BM25)
- Lección 12 — Query expansion y reformulación de consultas
- Lección 13 — Recuperación multi-hop
- Lección 14 — Filtros de contexto y ventana de recuperación
- Lección 15 — Recuperación con múltiples índices (routing)

Módulo 4: Generación y síntesis de respuestas

- Lección 16 — Construcción del prompt final con contexto recuperado
- Lección 17 — Manejo de contexto que no cabe en la ventana del modelo
- Lección 18 — Citas y trazabilidad de fuentes en la respuesta
- Lección 19 — Manejo de 'no sé' — evitar alucinaciones sin contexto suficiente
- Lección 20 — RAG conversacional — memoria y contexto multi-turno

Módulo 5: Evaluación y calidad de RAG

- Lección 21 — Métricas de evaluación de RAG — faithfulness, relevance
- Lección 22 — RAGAS y frameworks de evaluación automatizada
- Lección 23 — Evaluación de la etapa de recuperación vs generación
- Lección 24 — Testing de regresión para sistemas RAG
- Lección 25 — Feedback de usuarios y mejora continua

Módulo 6: Producción avanzada

Lección 26 — RAG multi-modal — imágenes y tablas

Lección 27 — Agentic RAG — recuperación como herramienta de un agente

Lección 28 — Costos de un sistema RAG a escala

Lección 29 — Seguridad — control de acceso a nivel de documento

Lección 30 — Proyecto final: sistema RAG completo listo para producción

Módulo 1: Arquitectura de sistemas RAG

Lección 01 — Anatomía de un pipeline RAG en producción

Ingesta y preprocesamiento de documentos, generación de embeddings, indexación vectorial, recuperación en tiempo de consulta, construcción del prompt, y generación de la respuesta final con el LLM.

Todo el trabajo costoso de procesar, generar embeddings e indexar los documentos ocurre de antemano en la fase offline; cuando llega una consulta real del usuario, el sistema solo necesita buscar en ese índice ya preparado, sin repetir ese procesamiento pesado en cada consulta individual.

Lección 02 — Ingesta de documentos — fuentes y formatos

Cada fuente puede tener su propio formato, estructura y mecanismo de acceso distintos, requiriendo conectores o adaptadores específicos para normalizar el contenido hacia un formato común antes de procesarlo uniformemente.

Una vez que el contenido de cualquier fuente (wiki, PDF, base de datos) se normaliza hacia el mismo formato de texto común, el resto del pipeline (chunking, generación de embeddings) puede aplicarse de la misma forma genérica, sin necesitar lógica separada y duplicada específica para cada tipo distinto de fuente original.

Lección 03 — Parsers y extracción de contenido (PDF, HTML, Office)

El PDF no tiene una estructura semántica explícita de párrafos y secciones como un formato de texto plano, requiriendo heurísticas o herramientas especializadas para reconstruir correctamente el orden de lectura y separar contenido de tablas del texto narrativo.

Un parser simple podría extraer el texto siguiendo el orden físico de las líneas en la página (de izquierda a derecha, de arriba a abajo) sin reconocer que el contenido está organizado en columnas separadas, resultando en fragmentos de texto de columnas distintas mezclados entre sí de forma incoherente, en vez de seguir el orden de lectura lógico real del documento.

Lección 04 — Preprocesamiento y limpieza de texto

Contenido irrelevante o repetitivo mezclado con el texto útil puede diluir el significado semántico capturado por el embedding, afectando negativamente la relevancia de las búsquedas posteriores.

El embedding se calcula a partir de todo el texto del fragmento, incluido cualquier contenido irrelevante mezclado como números de página o encabezados repetidos; esa información sin valor semántico real diluye ligeramente la representación vectorial resultante, alejándola de capturar fielmente solo el significado del contenido realmente útil del fragmento.

Lección 05 — Metadata y enriquecimiento de fragmentos

Permite filtrar o priorizar resultados de búsqueda según criterios estructurados adicionales (como solo documentos recientes, o de una categoría específica), combinando la búsqueda semántica con filtros precisos y confiables.

La similitud semántica mide qué tan relacionado está el contenido con la consulta, pero no garantiza por sí sola que ese contenido sea reciente; un filtro explícito de metadata por fecha da un criterio verificable y confiable para priorizar información actualizada, complementando lo que la búsqueda semántica sola no

podría garantizar.

Módulo 2: Bases de datos vectoriales

Lección 06 — Comparación de bases de datos vectoriales (Pinecone, Weaviate, Qdrant, pgvector)

Permite añadir capacidades de búsqueda vectorial directamente dentro de la infraestructura de PostgreSQL ya existente y operada por el equipo, sin introducir un sistema completamente nuevo y separado que aprender y mantener.

El equipo puede aprovechar directamente su experiencia operativa ya existente con PostgreSQL (backups, monitoreo, alta disponibilidad ya resueltos), evitando la complejidad adicional de aprender y mantener un sistema de base de datos vectorial completamente nuevo y separado, aunque eso implique renunciar a cierto rendimiento máximo que una solución dedicada podría ofrecer a escala muy grande.

Lección 07 — Índices ANN — HNSW y IVF

Reducen drásticamente el tiempo de búsqueda al evitar comparar la consulta contra absolutamente todos los vectores almacenados, a cambio de una pequeña probabilidad de no encontrar el resultado exactamente más cercano posible.

Comparar la consulta contra cada uno de millones de vectores almacenados individualmente tomaría un tiempo proporcional al tamaño total de la colección, resultando en una latencia inaceptable para una respuesta en tiempo real; el índice ANN acelera dramáticamente esa búsqueda a cambio de una pequeña probabilidad de no encontrar el resultado exactamente más cercano posible.

Lección 08 — Filtrado por metadata en búsquedas vectoriales

El índice ANN puede necesitar estrategias específicas para aplicar el filtro sin degradar significativamente la velocidad o precisión de la búsqueda, ya que filtrar después de buscar podría descartar resultados relevantes que el filtro habría excluido antes de completar la búsqueda por similitud.

Si la búsqueda vectorial trae los mejores candidatos semánticos sin considerar todavía el filtro de metadata, y luego se descartan los que no cumplen ese filtro, podría quedar muy pocos (o ningún) resultado final útil si la mayoría de los mejores candidatos semánticos originales no pertenecían a la categoría filtrada, requiriendo estrategias más sofisticadas para evitar ese problema.

Lección 09 — Namespaces y multi-tenancy en bases vectoriales

Aísla los documentos y embeddings de cada cliente entre sí, garantizando que una búsqueda de un cliente nunca devuelva accidentalmente contenido perteneciente a otro cliente distinto.

Sin esa separación explícita, todos los embeddings de todos los clientes vivirían mezclados en el mismo espacio de búsqueda compartido; una consulta de similitud vectorial simplemente encontraría los vectores más cercanos sin distinguir a qué cliente pertenecen, pudiendo devolver accidentalmente contenido confidencial de otro cliente completamente distinto al que hizo la consulta.

Lección 10 — Escalado y sharding de índices vectoriales

Dividir (sharding) el índice completo en múltiples particiones distribuidas entre distintos servidores, de forma que cada shard maneje solo un subconjunto de los vectores totales.

Como los vectores relevantes para una consulta específica podrían estar distribuidos en cualquiera de los distintos shards (no se sabe de antemano en cuál), es necesario consultar todos los shards relevantes y

luego combinar y reordenar los resultados parciales de cada uno, una coordinación adicional que no existía cuando todo vivía en un único servidor centralizado.

Módulo 3: Búsqueda híbrida y recuperación avanzada

Lección 11 — Búsqueda híbrida — combinar vectorial y léxica (BM25)

La búsqueda semántica puede pasar por alto coincidencias exactas importantes (como un código de producto específico o un término técnico exacto) que una búsqueda léxica encontraría directamente por coincidencia de palabras.

Un código de producto específico como 'SKU-4471-X' no tiene realmente un 'significado semántico' rico que un embedding pueda capturar de forma útil; una búsqueda léxica que busca la coincidencia exacta de esos caracteres específicos encontraría ese documento de forma mucho más directa y confiable que depender de la similitud semántica vectorial para ese tipo de término técnico preciso.

Lección 12 — Query expansion y reformulación de consultas

Generar variaciones o términos relacionados adicionales a partir de la consulta original (posiblemente usando el propio LLM), para capturar documentos relevantes que usen un vocabulario distinto al literal de la pregunta del usuario.

El usuario formula su pregunta con lenguaje cotidiano, mientras el documento relevante usa terminología técnica más formal y distinta; expandir la consulta original generando términos relacionados más técnicos ayuda a cerrar esa brecha de vocabulario, aumentando la probabilidad de encontrar el documento realmente relevante que de otra forma no coincidiría directamente con las palabras literales originales del usuario.

Lección 13 — Recuperación multi-hop

Una pregunta cuya respuesta completa requiere combinar información de múltiples documentos distintos relacionados entre sí, donde la respuesta a una primera parte de la pregunta es necesaria para saber qué buscar en un segundo paso.

Sin saber primero en qué departamento específico trabaja Juan (una información que hay que recuperar en un primer paso), sería imposible buscar directamente la política de vacaciones correcta correspondiente a ese departamento en particular, requiriendo encadenar dos búsquedas relacionadas en secuencia para llegar a la respuesta completa.

Lección 14 — Filtros de contexto y ventana de recuperación

El número máximo de fragmentos más relevantes (según la búsqueda de similitud) que se incluirán como contexto para que el LLM genere la respuesta final.

Si la respuesta completa a la pregunta requiere combinar información distribuida en varios fragmentos distintos del documento, pero el top-k configurado solo permite recuperar uno o dos fragmentos, el LLM podría quedarse sin suficiente contexto relevante disponible para generar una respuesta realmente completa y precisa.

Lección 15 — Recuperación con múltiples índices (routing)

Analiza la consulta del usuario para determinar a qué dominio o categoría de conocimiento pertenece (como 'documentación técnica' vs 'políticas de RRHH'), dirigiendo la búsqueda específicamente hacia el índice más apropiado en vez de buscar en todos indiscriminadamente.

Buscar indiscriminadamente en todos los índices disponibles (incluidos los que claramente no tienen relación con la consulta, como documentación técnica para una pregunta de RRHH) desperdiciaría tiempo de búsqueda y podría introducir resultados irrelevantes en la mezcla; dirigir la consulta específicamente hacia el índice correcto mejora tanto la velocidad como la relevancia final de los resultados obtenidos.

Módulo 4: Generación y síntesis de respuestas

Lección 16 — Construcción del prompt final con contexto recuperado

Instrucciones del sistema sobre cómo usar el contexto proporcionado, los fragmentos de texto recuperados relevantes, y la pregunta original del usuario, estructurados de forma clara para que el modelo distinga cada parte.

Si el contexto recuperado y la pregunta del usuario se mezclaran sin ninguna estructura clara dentro del prompt, el modelo podría tener más dificultad distinguiendo qué información es contexto de referencia y cuál es específicamente la pregunta que debe responder, mientras una estructura clara y etiquetada facilita que el modelo use correctamente cada parte según su función real.

Lección 17 — Manejo de contexto que no cabe en la ventana del modelo

Priorizar y truncar los fragmentos según su relevancia (posiblemente usando re-ranking), incluyendo solo los más importantes hasta llenar el espacio disponible dentro de la ventana de contexto del modelo.

Intentar comprimir todo el contenido recuperado de alguna forma genérica podría perder información importante de manera impredecible y difícil de controlar; priorizar explícitamente los fragmentos más relevantes según su importancia real y excluir los menos relevantes dentro del espacio disponible da un resultado más controlado y confiable que comprimir indiscriminadamente todo el contexto original.

Lección 18 — Citas y trazabilidad de fuentes en la respuesta

Permite al usuario verificar la fuente original de la información, aumentando la confianza en la respuesta y facilitando detectar si el modelo generó algo no respaldado realmente por el contexto recuperado.

Si el usuario puede revisar directamente el documento citado y no encuentra ahí la afirmación específica que la respuesta le atribuyó, eso revela claramente que el modelo generó esa información sin respaldo real (una alucinación), algo mucho más difícil de detectar sin ninguna cita verificable que conecte la afirmación con su supuesta fuente original.

Lección 19 — Manejo de 'no sé' — evitar alucinaciones sin contexto suficiente

Sin esa instrucción explícita, el modelo podría 'rellenar' la falta de información real con contenido inventado (alucinación) que suena convincente pero no está respaldado por ningún documento real del sistema.

Una respuesta generada con apariencia de confianza total, aunque en realidad no esté respaldada por ningún contexto relevante recuperado, es mucho más difícil de distinguir por el usuario de una respuesta genuinamente correcta, aumentando el riesgo de que confíe en información inventada sin darse cuenta de esa falta real de respaldo, comparado con un sistema que honestamente reconoce cuando no tiene suficiente información disponible.

Lección 20 — RAG conversacional — memoria y contexto multi-turno

Necesita considerar el historial previo de la conversación al reformular la consulta de recuperación, ya que una pregunta de seguimiento (como '¿y en el segundo caso?') solo tiene sentido en el contexto de lo que se preguntó antes.

La palabra 'eso' en esa pregunta de seguimiento no tiene ningún significado propio por sí sola; solo cobra sentido al conectarla con lo que se discutió en el turno anterior de la conversación, así que el sistema de

recuperación necesitaría reformular esa pregunta incorporando esa referencia explícita del contexto previo para poder buscar información realmente relevante.

Módulo 5: Evaluación y calidad de RAG

Lección 21 — Métricas de evaluación de RAG — faithfulness, relevance

Qué tan bien la respuesta generada se apega exclusivamente a la información realmente presente en el contexto recuperado, sin introducir afirmaciones inventadas o no respaldadas por ese contexto.

Relevance evalúa si el sistema de recuperación encontró los fragmentos correctos y pertinentes a la pregunta, mientras faithfulness evalúa una etapa completamente distinta y posterior: si el LLM, al generar la respuesta final, realmente se apegó fielmente a ese contexto correcto recuperado o si de todas formas introdujo contenido inventado no respaldado por él, siendo posible que una etapa funcione bien mientras la otra falle independientemente.

Lección 22 — RAGAS y frameworks de evaluación automatizada

Calcula automáticamente métricas estándar (como faithfulness, relevance, precision de recuperación) usando técnicas como LLM-as-judge, sin requerir que un humano evalúe manualmente cada respuesta individual generada por el sistema.

Evaluar manualmente cientos de respuestas distintas de forma consistente y rigurosa requeriría una cantidad significativa de tiempo humano; automatizar ese proceso con un framework como RAGAS permite evaluar sistemáticamente un volumen mucho mayor de casos de prueba, de forma más rápida y con un criterio consistente entre todas las evaluaciones.

Lección 23 — Evaluación de la etapa de recuperación vs generación

Permite identificar específicamente en cuál de las dos etapas está el problema si la respuesta final es de mala calidad, dirigiendo el esfuerzo de mejora hacia la parte del sistema que realmente lo necesita.

Si el documento verdaderamente relevante nunca llegó a formar parte del contexto proporcionado al modelo (porque la recuperación falló en encontrarlo), ningún ajuste al prompt de generación podría compensar esa ausencia fundamental de información correcta; el esfuerzo de mejora debería dirigirse específicamente hacia la etapa de recuperación, que es donde realmente está el problema raíz.

Lección 24 — Testing de regresión para sistemas RAG

Que el cambio no haya degradado la calidad de recuperación o generación en preguntas que antes funcionaban correctamente, comparando los resultados nuevos contra un baseline conocido del golden set de evaluación.

Un modelo de embeddings distinto produce representaciones vectoriales diferentes para exactamente el mismo contenido textual, lo que puede cambiar significativamente qué documentos se consideran más similares a una consulta dada; ese cambio fundamental en cómo se representa y busca todo el contenido justifica volver a validar el comportamiento completo del sistema mediante el testing de regresión.

Lección 25 — Feedback de usuarios y mejora continua

Provee señales reales sobre qué preguntas el sistema responde bien o mal en la práctica, identificando patrones de fallos específicos que la evaluación offline con un golden set podría no haber anticipado.

Los usuarios reales en producción formulan preguntas con una variedad y creatividad que el equipo probablemente no anticipó completamente al diseñar el golden set original limitado; el feedback negativo capturado en producción revela patrones genuinamente nuevos de fallos que solo se hacen evidentes con

el uso real a gran escala, alimentando la mejora continua del sistema con casos concretos antes desconocidos.

Módulo 6: Producción avanzada

Lección 26 — RAG multi-modal — imágenes y tablas

Requiere generar representaciones (embeddings) apropiadas para contenido no textual, o convertir ese contenido visual/tabular hacia una descripción textual procesable, dado que los modelos de embeddings tradicionales están diseñados principalmente para texto.

Una tabla tiene una estructura bidimensional real donde el significado de cada valor depende de su fila y columna específica correspondiente; simplemente extraer el texto de forma lineal sin preservar esa estructura relacional perdería información importante sobre qué dato específico corresponde a qué categoría, requiriendo un tratamiento más cuidadoso para mantener ese significado estructural original al procesarla.

Lección 27 — Agentic RAG — recuperación como herramienta de un agente

El LLM actúa como un agente que decide de forma autónoma cuándo y cuántas veces invocar la recuperación como una herramienta disponible, pudiendo hacer múltiples búsquedas iterativas según lo que va descubriendo, en vez de un único paso de recuperación fijo predeterminado.

Una única búsqueda inicial fija podría no anticipar todos los subtemas relacionados que realmente se necesitan investigar para responder completamente una pregunta compleja; un agente que puede decidir hacer búsquedas adicionales a medida que va descubriendo qué información le falta tiene la flexibilidad de profundizar iterativamente en cada subtema relevante, algo que un único paso de recuperación predeterminado no podría lograr de la misma forma.

Lección 28 — Costos de un sistema RAG a escala

El costo de generar embeddings (tanto en la indexación inicial como potencialmente en cada consulta), el costo de almacenamiento y operación de la base de datos vectorial, y el costo de cómputo del propio proceso de recuperación.

Mantener una base de datos vectorial con millones de fragmentos indexados requiere infraestructura de almacenamiento y cómputo proporcional al tamaño real de esa colección, un costo que existe de forma más o menos constante independientemente del volumen específico de consultas diarias, siendo un componente de costo separado y significativo además del costo variable por cada consulta individual procesada.

Lección 29 — Seguridad — control de acceso a nivel de documento

Evita que un usuario con permisos limitados reciba en su respuesta información proveniente de documentos confidenciales a los que no debería tener acceso, aunque esos documentos fueran semánticamente relevantes a su consulta.

El LLM no tiene ninguna forma confiable e inherente de saber qué nivel de permiso tiene el usuario específico haciendo la consulta actual, así que confiar en que el modelo 'decida' correctamente no usar información sensible no es una garantía de seguridad real; el filtro de permisos debe aplicarse antes, en la etapa de recuperación, para que ese contenido restringido simplemente nunca llegue a formar parte del contexto que el modelo recibe en primer lugar.

Lección 30 — Proyecto final: sistema RAG completo listo para producción

Ingesta y preprocesamiento robusto de documentos + búsqueda híbrida con re-ranking + manejo correcto de contexto y citas + evaluación continua con RAGAS/golden set + control de acceso por documento + monitoreo de costos y calidad.

Intenta mencionar al menos dos pilares: ingesta y preprocesamiento robusto, búsqueda híbrida con re-ranking, evaluación continua con RAGAS o control de acceso por documento.